ЛР4

SQL-скрипт создания схемы:

```
CREATE TABLE bakery (
    bakery_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL,
    address text NOT NULL,
    open_time time NOT NULL,
    close_time time NOT NULL,
    rent_cost numeric(12,2) NOT NULL CHECK (rent_cost >= 0),
    phone text,
    CONSTRAINT chk_bakery_work_time CHECK (close_time > open_time)
);

CREATE TABLE client (
    client_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name text NOT NULL,
    phone text NOT NULL,
    email text,
    default_delivery_address text
);

CREATE TABLE employee (
    employee_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    bakery_id bigint NOT NULL,
    full_name text NOT NULL,
    passport_no text NOT NULL,
    role_code text NOT NULL,
    hourly_rate numeric(12,2) NOT NULL DEFAULT 0 CHECK (hourly_rate >= 0),
    fixed_monthly_salary numeric(12,2) NOT NULL DEFAULT 0 CHECK
(fixed_monthly_salary >= 0),
```

```
    courier_bonus_pct numeric(5,2) NOT NULL DEFAULT 0 CHECK (courier_bonus_pct >= 0
AND courier_bonus_pct <= 100),

    min_hours_per_month numeric(6,2) NOT NULL DEFAULT 0 CHECK
(min_hours_per_month >= 0),

    login text NOT NULL UNIQUE,

    password_hash text NOT NULL,

    is_active boolean NOT NULL DEFAULT true,

    hire_date date NOT NULL,

    CONSTRAINT uq_employee_passport UNIQUE (passport_no),

    CONSTRAINT chk_employee_role

        CHECK (role_code IN ('CHEF', 'CONFECTIONER', 'CASHIER', 'COURIER',
'OWNER')),

    CONSTRAINT fk_employee_bakery

        FOREIGN KEY (bakery_id)

        REFERENCES bakery(bakery_id)

        ON DELETE RESTRICT

        ON UPDATE CASCADE

);


CREATE TABLE work_schedule (

    schedule_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    employee_id bigint NOT NULL,

    bakery_id bigint NOT NULL,

    work_date date NOT NULL,

    day_of_week smallint NOT NULL,

    work_start_time time NOT NULL,

    work_end_time time NOT NULL,

    break_start_time time,

    break_end_time time,

    CONSTRAINT chk_schedule_day_of_week CHECK (day_of_week BETWEEN 1 AND 7),

    CONSTRAINT chk_schedule_work_time CHECK (work_end_time > work_start_time),

    CONSTRAINT chk_schedule_break_time CHECK (

        (break_start_time IS NULL AND break_end_time IS NULL)
```

```
        OR
        (break_start_time IS NOT NULL AND break_end_time IS NOT NULL
         AND break_start_time >= work_start_time
         AND break_end_time <= work_end_time
         AND break_end_time > break_start_time)
    ),
    CONSTRAINT fk_schedule_employee
        FOREIGN KEY (employee_id)
        REFERENCES employee(employee_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_schedule_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE ingredient (
    ingredient_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL,
    description text,
    cost_per_kg numeric(12,2) CHECK (cost_per_kg IS NULL OR cost_per_kg >= 0),
    cost_per_piece numeric(12,2) CHECK (cost_per_piece IS NULL OR cost_per_piece >= 0),
    calories_per_100g numeric(8,2) CHECK (calories_per_100g IS NULL OR calories_per_100g
>= 0),
    calories_per_piece numeric(8,2) CHECK (calories_per_piece IS NULL OR
calories_per_piece >= 0),
    shelf_life_days integer CHECK (shelf_life_days IS NULL OR shelf_life_days >= 0),
    can_be_weight boolean NOT NULL DEFAULT false,
    can_be_piece boolean NOT NULL DEFAULT false,
    CONSTRAINT chk_ingredient_measure CHECK (can_be_weight OR can_be_piece)
```

```
);

CREATE TABLE utensil (
    utensil_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL,
    description text
);

CREATE TABLE store (
    store_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL,
    address text NOT NULL,
    phone text
);

CREATE TABLE product (
    product_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    chef_id bigint NOT NULL,
    name text NOT NULL,
    recipe_description text,
    is_active boolean NOT NULL DEFAULT true,
    CONSTRAINT fk_product_chef
        FOREIGN KEY (chef_id)
        REFERENCES employee(employee_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE bakery_ingredient_stock (
    bakery_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    stock_qty_g numeric(12,3) NOT NULL DEFAULT 0 CHECK (stock_qty_g >= 0),
```

```
    stock_qty_pcs integer NOT NULL DEFAULT 0 CHECK (stock_qty_pcs >= 0),
    avg_daily_consumption_g numeric(12,3) NOT NULL DEFAULT 0 CHECK
(avg_daily_consumption_g >= 0),
    avg_daily_consumption_pcs integer NOT NULL DEFAULT 0 CHECK
(avg_daily_consumption_pcs >= 0),
    CONSTRAINT pk_bakery_ingredient_stock PRIMARY KEY (bakery_id, ingredient_id),
    CONSTRAINT fk_bakery_ingredient_stock_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_bakery_ingredient_stock_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE bakery_utensil_inventory (
    bakery_id bigint NOT NULL,
    utensil_id bigint NOT NULL,
    qty_available integer NOT NULL DEFAULT 0 CHECK (qty_available >= 0),
    condition_status text,
    CONSTRAINT pk_bakery_utensil_inventory PRIMARY KEY (bakery_id, utensil_id),
    CONSTRAINT fk_bakery_utensil_inventory_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_bakery_utensil_inventory_utensil
        FOREIGN KEY (utensil_id)
        REFERENCES utensil(utensil_id)
```

```
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE store_ingredient_assortment (
    store_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    price_per_kg numeric(12,2) CHECK (price_per_kg IS NULL OR price_per_kg >= 0),
    price_per_piece numeric(12,2) CHECK (price_per_piece IS NULL OR price_per_piece >= 0),
    stock_qty_g numeric(12,3) NOT NULL DEFAULT 0 CHECK (stock_qty_g >= 0),
    stock_qty_pcs integer NOT NULL DEFAULT 0 CHECK (stock_qty_pcs >= 0),
    CONSTRAINT pk_store_ingredient_assortment PRIMARY KEY (store_id, ingredient_id),
    CONSTRAINT fk_store_ingredient_assortment_store
        FOREIGN KEY (store_id)
        REFERENCES store(store_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_store_ingredient_assortment_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE store_utensil_assortment (
    store_id bigint NOT NULL,
    utensil_id bigint NOT NULL,
    price_amount numeric(12,2) NOT NULL CHECK (price_amount >= 0),
    stock_qty integer NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
    CONSTRAINT pk_store_utensil_assortment PRIMARY KEY (store_id, utensil_id),
    CONSTRAINT fk_store_utensil_assortment_store
        FOREIGN KEY (store_id)
```

```
        REFERENCES store(store_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_store_utensil_assortment_utensil
        FOREIGN KEY (utensil_id)
        REFERENCES utensil(utensil_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE product_utensil (
    product_id bigint NOT NULL,
    utensil_id bigint NOT NULL,
    usage_note text,
    CONSTRAINT pk_product_utensil PRIMARY KEY (product_id, utensil_id),
    CONSTRAINT fk_product_utensil_product
        FOREIGN KEY (product_id)
        REFERENCES product(product_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_product_utensil_utensil
        FOREIGN KEY (utensil_id)
        REFERENCES utensil(utensil_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE price_list (
    price_list_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    bakery_id bigint NOT NULL,
    name text NOT NULL,
    valid_from date NOT NULL,
```

```
    valid_to date,
    is_active boolean NOT NULL DEFAULT true,
    CONSTRAINT chk_price_list_dates CHECK (valid_to IS NULL OR valid_to >= valid_from),
    CONSTRAINT fk_price_list_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE price_list_item (
    price_list_item_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    price_list_id bigint NOT NULL,
    product_id bigint NOT NULL,
    item_code text NOT NULL,
    sale_price numeric(12,2) NOT NULL CHECK (sale_price >= 0),
    prep_time_minutes integer NOT NULL CHECK (prep_time_minutes >= 0),
    calories_kcal numeric(8,2) NOT NULL CHECK (calories_kcal >= 0),
    CONSTRAINT uq_price_list_item_code UNIQUE (price_list_id, item_code),
    CONSTRAINT fk_price_list_item_price_list
        FOREIGN KEY (price_list_id)
        REFERENCES price_list(price_list_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_price_list_item_product
        FOREIGN KEY (product_id)
        REFERENCES product(product_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

```
CREATE TABLE price_list_item_ingredient (
    price_list_item_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    amount_value numeric(12,3) NOT NULL CHECK (amount_value > 0),
    amount_unit text NOT NULL,
    taste_contribution_pct numeric(5,2) NOT NULL CHECK (taste_contribution_pct >= 0 AND taste_contribution_pct <= 100),
    CONSTRAINT pk_price_list_item_ingredient PRIMARY KEY (price_list_item_id, ingredient_id),
    CONSTRAINT chk_price_list_item_ingredient_unit CHECK (amount_unit IN ('g', 'pcs')),
    CONSTRAINT fk_price_list_item_ingredient_item
        FOREIGN KEY (price_list_item_id)
        REFERENCES price_list_item(price_list_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_price_list_item_ingredient_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE allowed_topping (
    price_list_item_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    default_amount numeric(12,3) NOT NULL CHECK (default_amount > 0),
    amount_unit text NOT NULL,
    extra_price numeric(12,2) NOT NULL CHECK (extra_price >= 0),
    taste_drop_pct numeric(5,2) NOT NULL CHECK (taste_drop_pct >= 0 AND taste_drop_pct <= 100),
    CONSTRAINT pk_allowed_topping PRIMARY KEY (price_list_item_id, ingredient_id),
    CONSTRAINT chk_allowed_topping_unit CHECK (amount_unit IN ('g', 'pcs')),
    CONSTRAINT fk_allowed_topping_item
```

```
        FOREIGN KEY (price_list_item_id)
        REFERENCES price_list_item(price_list_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_allowed_topping_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE replacement_group (
    replacement_group_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    price_list_item_id bigint NOT NULL,
    group_name text NOT NULL,
    comment text,
    CONSTRAINT fk_replacement_group_item
        FOREIGN KEY (price_list_item_id)
        REFERENCES price_list_item(price_list_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE replacement_group_item (
    replacement_group_id bigint NOT NULL,
    base_ingredient_id bigint NOT NULL,
    substitute_ingredient_id bigint NOT NULL,
    taste_drop_pct numeric(5,2) NOT NULL CHECK (taste_drop_pct >= 0 AND taste_drop_pct
<= 100),
    CONSTRAINT pk_replacement_group_item PRIMARY KEY (
        replacement_group_id,
        base_ingredient_id,
```

```
        substitute_ingredient_id
    ),
    CONSTRAINT chk_replacement_not_same CHECK (base_ingredient_id <>
substitute_ingredient_id),
    CONSTRAINT fk_replacement_group_item_group
        FOREIGN KEY (replacement_group_id)
        REFERENCES replacement_group(replacement_group_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_replacement_group_item_base_ingredient
        FOREIGN KEY (base_ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_replacement_group_item_substitute_ingredient
        FOREIGN KEY (substitute_ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);


CREATE TABLE customer_order (
    order_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    client_id bigint NOT NULL,
    bakery_id bigint NOT NULL,
    cashier_id bigint NOT NULL,
    order_datetime timestamptz NOT NULL DEFAULT now(),
    order_type text NOT NULL,
    status text NOT NULL,
    order_total_amount numeric(12,2) NOT NULL DEFAULT 0 CHECK (order_total_amount
>= 0),
    CONSTRAINT fk_customer_order_client
```

```
    FOREIGN KEY (client_id)
    REFERENCES client(client_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT fk_customer_order_bakery
    FOREIGN KEY (bakery_id)
    REFERENCES bakery(bakery_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,
  CONSTRAINT fk_customer_order_cashier
    FOREIGN KEY (cashier_id)
    REFERENCES employee(employee_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

CREATE TABLE invoice (
  invoice_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id bigint NOT NULL,
  courier_employee_id bigint NOT NULL,
  recipient_fio text NOT NULL,
  delivery_address text NOT NULL,
  delivery_due_at timestamptz NOT NULL,
  delivered_at timestamptz,
  recipient_signed boolean NOT NULL DEFAULT false,
  claim_text text,
  has_claim boolean NOT NULL DEFAULT false,
  CONSTRAINT uq_invoice_order UNIQUE (order_id),
  CONSTRAINT fk_invoice_order
    FOREIGN KEY (order_id)
    REFERENCES customer_order(order_id)
    ON DELETE CASCADE
```

```
        ON UPDATE CASCADE,
    CONSTRAINT fk_invoice_courier
        FOREIGN KEY (courier_employee_id)
        REFERENCES employee(employee_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE order_item (
    order_item_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id bigint NOT NULL,
    price_list_item_id bigint NOT NULL,
    quantity integer NOT NULL CHECK (quantity > 0),
    base_price numeric(12,2) NOT NULL CHECK (base_price >= 0),
    toppings_price numeric(12,2) NOT NULL DEFAULT 0 CHECK (toppings_price >= 0),
    line_total_amount numeric(12,2) NOT NULL CHECK (line_total_amount >= 0),
    final_taste_pct numeric(5,2) NOT NULL CHECK (final_taste_pct >= 0 AND final_taste_pct
<= 100),
    CONSTRAINT fk_order_item_order
        FOREIGN KEY (order_id)
        REFERENCES customer_order(order_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_order_item_price_list_item
        FOREIGN KEY (price_list_item_id)
        REFERENCES price_list_item(price_list_item_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE order_item_topping (
    order_item_id bigint NOT NULL,
```

```
    ingredient_id bigint NOT NULL,
    amount_value numeric(12,3) NOT NULL CHECK (amount_value > 0),
    amount_unit text NOT NULL,
    extra_price numeric(12,2) NOT NULL CHECK (extra_price >= 0),
    taste_drop_pct_applied numeric(5,2) NOT NULL CHECK (taste_drop_pct_applied >= 0
AND taste_drop_pct_applied <= 100),
    CONSTRAINT pk_order_item_topping PRIMARY KEY (order_item_id, ingredient_id),
    CONSTRAINT chk_order_item_topping_unit CHECK (amount_unit IN ('g', 'pcs')),
    CONSTRAINT fk_order_item_topping_order_item
        FOREIGN KEY (order_item_id)
        REFERENCES order_item(order_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_order_item_topping_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE order_item_exclusion (
    order_item_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT pk_order_item_exclusion PRIMARY KEY (order_item_id, ingredient_id),
    CONSTRAINT fk_order_item_exclusion_order_item
        FOREIGN KEY (order_item_id)
        REFERENCES order_item(order_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_order_item_exclusion_ingredient
        FOREIGN KEY (ingredient_id)
```

```
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE invoice_return_item (
    invoice_id bigint NOT NULL,
    order_item_id bigint NOT NULL,
    is_returned boolean NOT NULL DEFAULT true,
    return_reason text,
    CONSTRAINT pk_invoice_return_item PRIMARY KEY (invoice_id, order_item_id),
    CONSTRAINT fk_invoice_return_item_invoice
        FOREIGN KEY (invoice_id)
        REFERENCES invoice(invoice_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_invoice_return_item_order_item
        FOREIGN KEY (order_item_id)
        REFERENCES order_item(order_item_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE activity_log (
    log_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    employee_id bigint NOT NULL,
    action_type text NOT NULL,
    action_at timestamptz NOT NULL DEFAULT now(),
    info text,
    CONSTRAINT fk_activity_log_employee
        FOREIGN KEY (employee_id)
        REFERENCES employee(employee_id)
```

```
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);
```

Скрипт тестового заполнения данными:

```
CREATE TABLE bakery (
    bakery_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL,
    address text NOT NULL,
    open_time time NOT NULL,
    close_time time NOT NULL,
    rent_cost numeric(12,2) NOT NULL CHECK (rent_cost >= 0),
    phone text,
    CONSTRAINT chk_bakery_work_time CHECK (close_time > open_time)
);

CREATE TABLE client (
    client_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name text NOT NULL,
    phone text NOT NULL,
    email text,
    default_delivery_address text
);

CREATE TABLE employee (
    employee_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    bakery_id bigint NOT NULL,
    full_name text NOT NULL,
    passport_no text NOT NULL,
    role_code text NOT NULL,
    hourly_rate numeric(12,2) NOT NULL DEFAULT 0 CHECK (hourly_rate >= 0),
```

```
    fixed_monthly_salary numeric(12,2) NOT NULL DEFAULT 0 CHECK
(fixed_monthly_salary >= 0),

    courier_bonus_pct numeric(5,2) NOT NULL DEFAULT 0 CHECK (courier_bonus_pct >= 0
AND courier_bonus_pct <= 100),

    min_hours_per_month numeric(6,2) NOT NULL DEFAULT 0 CHECK
(min_hours_per_month >= 0),

    login text NOT NULL UNIQUE,

    password_hash text NOT NULL,

    is_active boolean NOT NULL DEFAULT true,

    hire_date date NOT NULL,

    CONSTRAINT uq_employee_passport UNIQUE (passport_no),

    CONSTRAINT chk_employee_role

        CHECK (role_code IN ('CHEF', 'CONFECTIONER', 'CASHIER', 'COURIER',
'OWNER')),

    CONSTRAINT fk_employee_bakery

        FOREIGN KEY (bakery_id)

        REFERENCES bakery(bakery_id)

        ON DELETE RESTRICT

        ON UPDATE CASCADE

);


CREATE TABLE work_schedule (

    schedule_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

    employee_id bigint NOT NULL,

    bakery_id bigint NOT NULL,

    work_date date NOT NULL,

    day_of_week smallint NOT NULL,

    work_start_time time NOT NULL,

    work_end_time time NOT NULL,

    break_start_time time,

    break_end_time time,

    CONSTRAINT chk_schedule_day_of_week CHECK (day_of_week BETWEEN 1 AND 7),

    CONSTRAINT chk_schedule_work_time CHECK (work_end_time > work_start_time),
```

```
    CONSTRAINT chk_schedule_break_time CHECK (
        (break_start_time IS NULL AND break_end_time IS NULL)
        OR
        (break_start_time IS NOT NULL AND break_end_time IS NOT NULL
         AND break_start_time >= work_start_time
         AND break_end_time <= work_end_time
         AND break_end_time > break_start_time)
    ),
    CONSTRAINT fk_schedule_employee
        FOREIGN KEY (employee_id)
        REFERENCES employee(employee_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_schedule_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);


CREATE TABLE ingredient (
    ingredient_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL,
    description text,
    cost_per_kg numeric(12,2) CHECK (cost_per_kg IS NULL OR cost_per_kg >= 0),
    cost_per_piece numeric(12,2) CHECK (cost_per_piece IS NULL OR cost_per_piece >= 0),
    calories_per_100g numeric(8,2) CHECK (calories_per_100g IS NULL OR calories_per_100g
>= 0),
    calories_per_piece numeric(8,2) CHECK (calories_per_piece IS NULL OR
calories_per_piece >= 0),
    shelf_life_days integer CHECK (shelf_life_days IS NULL OR shelf_life_days >= 0),
    can_be_weight boolean NOT NULL DEFAULT false,
```

```
    can_be_piece boolean NOT NULL DEFAULT false,
    CONSTRAINT chk_ingredient_measure CHECK (can_be_weight OR can_be_piece)
);

CREATE TABLE utensil (
    utensil_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL,
    description text
);

CREATE TABLE store (
    store_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name text NOT NULL,
    address text NOT NULL,
    phone text
);

CREATE TABLE product (
    product_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    chef_id bigint NOT NULL,
    name text NOT NULL,
    recipe_description text,
    is_active boolean NOT NULL DEFAULT true,
    CONSTRAINT fk_product_chef
        FOREIGN KEY (chef_id)
        REFERENCES employee(employee_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE bakery_ingredient_stock (
    bakery_id bigint NOT NULL,
```

```
    ingredient_id bigint NOT NULL,
    stock_qty_g numeric(12,3) NOT NULL DEFAULT 0 CHECK (stock_qty_g >= 0),
    stock_qty_pcs integer NOT NULL DEFAULT 0 CHECK (stock_qty_pcs >= 0),
    avg_daily_consumption_g numeric(12,3) NOT NULL DEFAULT 0 CHECK
(avg_daily_consumption_g >= 0),
    avg_daily_consumption_pcs integer NOT NULL DEFAULT 0 CHECK
(avg_daily_consumption_pcs >= 0),
    CONSTRAINT pk_bakery_ingredient_stock PRIMARY KEY (bakery_id, ingredient_id),
    CONSTRAINT fk_bakery_ingredient_stock_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_bakery_ingredient_stock_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE bakery_utensil_inventory (
    bakery_id bigint NOT NULL,
    utensil_id bigint NOT NULL,
    qty_available integer NOT NULL DEFAULT 0 CHECK (qty_available >= 0),
    condition_status text,
    CONSTRAINT pk_bakery_utensil_inventory PRIMARY KEY (bakery_id, utensil_id),
    CONSTRAINT fk_bakery_utensil_inventory_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_bakery_utensil_inventory_utensil
```

```
    FOREIGN KEY (utensil_id)
    REFERENCES utensil(utensil_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

CREATE TABLE store_ingredient_assortment (
  store_id bigint NOT NULL,
  ingredient_id bigint NOT NULL,
  price_per_kg numeric(12,2) CHECK (price_per_kg IS NULL OR price_per_kg >= 0),
  price_per_piece numeric(12,2) CHECK (price_per_piece IS NULL OR price_per_piece >= 0),
  stock_qty_g numeric(12,3) NOT NULL DEFAULT 0 CHECK (stock_qty_g >= 0),
  stock_qty_pcs integer NOT NULL DEFAULT 0 CHECK (stock_qty_pcs >= 0),
  CONSTRAINT pk_store_ingredient_assortment PRIMARY KEY (store_id, ingredient_id),
  CONSTRAINT fk_store_ingredient_assortment_store
    FOREIGN KEY (store_id)
    REFERENCES store(store_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_store_ingredient_assortment_ingredient
    FOREIGN KEY (ingredient_id)
    REFERENCES ingredient(ingredient_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

CREATE TABLE store_utensil_assortment (
  store_id bigint NOT NULL,
  utensil_id bigint NOT NULL,
  price_amount numeric(12,2) NOT NULL CHECK (price_amount >= 0),
  stock_qty integer NOT NULL DEFAULT 0 CHECK (stock_qty >= 0),
  CONSTRAINT pk_store_utensil_assortment PRIMARY KEY (store_id, utensil_id),
```

```
    CONSTRAINT fk_store_utensil_assortment_store
        FOREIGN KEY (store_id)
        REFERENCES store(store_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_store_utensil_assortment_utensil
        FOREIGN KEY (utensil_id)
        REFERENCES utensil(utensil_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE product_utensil (
    product_id bigint NOT NULL,
    utensil_id bigint NOT NULL,
    usage_note text,
    CONSTRAINT pk_product_utensil PRIMARY KEY (product_id, utensil_id),
    CONSTRAINT fk_product_utensil_product
        FOREIGN KEY (product_id)
        REFERENCES product(product_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_product_utensil_utensil
        FOREIGN KEY (utensil_id)
        REFERENCES utensil(utensil_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE price_list (
    price_list_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    bakery_id bigint NOT NULL,
```

```
    name text NOT NULL,
    valid_from date NOT NULL,
    valid_to date,
    is_active boolean NOT NULL DEFAULT true,
    CONSTRAINT chk_price_list_dates CHECK (valid_to IS NULL OR valid_to >=
valid_from),
    CONSTRAINT fk_price_list_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);


CREATE TABLE price_list_item (
    price_list_item_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    price_list_id bigint NOT NULL,
    product_id bigint NOT NULL,
    item_code text NOT NULL,
    sale_price numeric(12,2) NOT NULL CHECK (sale_price >= 0),
    prep_time_minutes integer NOT NULL CHECK (prep_time_minutes >= 0),
    calories_kcal numeric(8,2) NOT NULL CHECK (calories_kcal >= 0),
    CONSTRAINT uq_price_list_item_code UNIQUE (price_list_id, item_code),
    CONSTRAINT fk_price_list_item_price_list
        FOREIGN KEY (price_list_id)
        REFERENCES price_list(price_list_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_price_list_item_product
        FOREIGN KEY (product_id)
        REFERENCES product(product_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
```

```
);

CREATE TABLE price_list_item_ingredient (
    price_list_item_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    amount_value numeric(12,3) NOT NULL CHECK (amount_value > 0),
    amount_unit text NOT NULL,
    taste_contribution_pct numeric(5,2) NOT NULL CHECK (taste_contribution_pct >= 0 AND
taste_contribution_pct <= 100),
    CONSTRAINT pk_price_list_item_ingredient PRIMARY KEY (price_list_item_id,
ingredient_id),
    CONSTRAINT chk_price_list_item_ingredient_unit CHECK (amount_unit IN ('g', 'pcs')),
    CONSTRAINT fk_price_list_item_ingredient_item
        FOREIGN KEY (price_list_item_id)
        REFERENCES price_list_item(price_list_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_price_list_item_ingredient_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE allowed_topping (
    price_list_item_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    default_amount numeric(12,3) NOT NULL CHECK (default_amount > 0),
    amount_unit text NOT NULL,
    extra_price numeric(12,2) NOT NULL CHECK (extra_price >= 0),
    taste_drop_pct numeric(5,2) NOT NULL CHECK (taste_drop_pct >= 0 AND taste_drop_pct
<= 100),
    CONSTRAINT pk_allowed_topping PRIMARY KEY (price_list_item_id, ingredient_id),
```

```
    CONSTRAINT chk_allowed_topping_unit CHECK (amount_unit IN ('g', 'pcs')),
    CONSTRAINT fk_allowed_topping_item
        FOREIGN KEY (price_list_item_id)
        REFERENCES price_list_item(price_list_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_allowed_topping_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE replacement_group (
    replacement_group_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    price_list_item_id bigint NOT NULL,
    group_name text NOT NULL,
    comment text,
    CONSTRAINT fk_replacement_group_item
        FOREIGN KEY (price_list_item_id)
        REFERENCES price_list_item(price_list_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

CREATE TABLE replacement_group_item (
    replacement_group_id bigint NOT NULL,
    base_ingredient_id bigint NOT NULL,
    substitute_ingredient_id bigint NOT NULL,
    taste_drop_pct numeric(5,2) NOT NULL CHECK (taste_drop_pct >= 0 AND taste_drop_pct
<= 100),
    CONSTRAINT pk_replacement_group_item PRIMARY KEY (
```

```
        replacement_group_id,
        base_ingredient_id,
        substitute_ingredient_id
    ),
    CONSTRAINT chk_replacement_not_same CHECK (base_ingredient_id <>
substitute_ingredient_id),
    CONSTRAINT fk_replacement_group_item_group
        FOREIGN KEY (replacement_group_id)
        REFERENCES replacement_group(replacement_group_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_replacement_group_item_base_ingredient
        FOREIGN KEY (base_ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_replacement_group_item_substitute_ingredient
        FOREIGN KEY (substitute_ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);


CREATE TABLE customer_order (
    order_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    client_id bigint NOT NULL,
    bakery_id bigint NOT NULL,
    cashier_id bigint NOT NULL,
    order_datetime timestamptz NOT NULL DEFAULT now(),
    order_type text NOT NULL,
    status text NOT NULL,
```

```
    order_total_amount numeric(12,2) NOT NULL DEFAULT 0 CHECK (order_total_amount
>= 0),
    CONSTRAINT fk_customer_order_client
        FOREIGN KEY (client_id)
        REFERENCES client(client_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_customer_order_bakery
        FOREIGN KEY (bakery_id)
        REFERENCES bakery(bakery_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE,
    CONSTRAINT fk_customer_order_cashier
        FOREIGN KEY (cashier_id)
        REFERENCES employee(employee_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);

CREATE TABLE invoice (
    invoice_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    order_id bigint NOT NULL,
    courier_employee_id bigint NOT NULL,
    recipient_fio text NOT NULL,
    delivery_address text NOT NULL,
    delivery_due_at timestamptz NOT NULL,
    delivered_at timestamptz,
    recipient_signed boolean NOT NULL DEFAULT false,
    claim_text text,
    has_claim boolean NOT NULL DEFAULT false,
    CONSTRAINT uq_invoice_order UNIQUE (order_id),
    CONSTRAINT fk_invoice_order
```

```
    FOREIGN KEY (order_id)
    REFERENCES customer_order(order_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_invoice_courier
    FOREIGN KEY (courier_employee_id)
    REFERENCES employee(employee_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

CREATE TABLE order_item (
  order_item_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id bigint NOT NULL,
  price_list_item_id bigint NOT NULL,
  quantity integer NOT NULL CHECK (quantity > 0),
  base_price numeric(12,2) NOT NULL CHECK (base_price >= 0),
  toppings_price numeric(12,2) NOT NULL DEFAULT 0 CHECK (toppings_price >= 0),
  line_total_amount numeric(12,2) NOT NULL CHECK (line_total_amount >= 0),
  final_taste_pct numeric(5,2) NOT NULL CHECK (final_taste_pct >= 0 AND final_taste_pct
<= 100),
  CONSTRAINT fk_order_item_order
    FOREIGN KEY (order_id)
    REFERENCES customer_order(order_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_order_item_price_list_item
    FOREIGN KEY (price_list_item_id)
    REFERENCES price_list_item(price_list_item_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);
```

```
CREATE TABLE order_item_topping (
    order_item_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    amount_value numeric(12,3) NOT NULL CHECK (amount_value > 0),
    amount_unit text NOT NULL,
    extra_price numeric(12,2) NOT NULL CHECK (extra_price >= 0),
    taste_drop_pct_applied numeric(5,2) NOT NULL CHECK (taste_drop_pct_applied >= 0
AND taste_drop_pct_applied <= 100),
    CONSTRAINT pk_order_item_topping PRIMARY KEY (order_item_id, ingredient_id),
    CONSTRAINT chk_order_item_topping_unit CHECK (amount_unit IN ('g', 'pcs')),
    CONSTRAINT fk_order_item_topping_order_item
        FOREIGN KEY (order_item_id)
        REFERENCES order_item(order_item_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE,
    CONSTRAINT fk_order_item_topping_ingredient
        FOREIGN KEY (ingredient_id)
        REFERENCES ingredient(ingredient_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);


CREATE TABLE order_item_exclusion (
    order_item_id bigint NOT NULL,
    ingredient_id bigint NOT NULL,
    created_at timestamptz NOT NULL DEFAULT now(),
    CONSTRAINT pk_order_item_exclusion PRIMARY KEY (order_item_id, ingredient_id),
    CONSTRAINT fk_order_item_exclusion_order_item
        FOREIGN KEY (order_item_id)
        REFERENCES order_item(order_item_id)
        ON DELETE CASCADE
```

```
    ON UPDATE CASCADE,
  CONSTRAINT fk_order_item_exclusion_ingredient
    FOREIGN KEY (ingredient_id)
    REFERENCES ingredient(ingredient_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

CREATE TABLE invoice_return_item (
  invoice_id bigint NOT NULL,
  order_item_id bigint NOT NULL,
  is_returned boolean NOT NULL DEFAULT true,
  return_reason text,
  CONSTRAINT pk_invoice_return_item PRIMARY KEY (invoice_id, order_item_id),
  CONSTRAINT fk_invoice_return_item_invoice
    FOREIGN KEY (invoice_id)
    REFERENCES invoice(invoice_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,
  CONSTRAINT fk_invoice_return_item_order_item
    FOREIGN KEY (order_item_id)
    REFERENCES order_item(order_item_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);

CREATE TABLE activity_log (
  log_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  employee_id bigint NOT NULL,
  action_type text NOT NULL,
  action_at timestamptz NOT NULL DEFAULT now(),
  info text,
```

```
    CONSTRAINT fk_activity_log_employee
        FOREIGN KEY (employee_id)
        REFERENCES employee(employee_id)
        ON DELETE RESTRICT
        ON UPDATE CASCADE
);
```

Скрит заполнения данными на python:

```
from __future__ import annotations

import argparse
import hashlib
import random
from dataclasses import dataclass
from datetime import date, datetime, time, timedelta
from decimal import Decimal, ROUND_HALF_UP

import psycopg2
from psycopg2.extras import execute_values

@dataclass(frozen=True)
class ScenarioConfig:
    name: str
    bakery_count: int
    clients_per_bakery: int
    confectioners_per_bakery: int
    cashiers_per_bakery: int
    couriers_per_bakery: int
    products_per_bakery: int
    orders_per_bakery: int
    schedule_days: int
    max_items_per_order: int
```

```
    delivery_share: float
    commit_every: int
    open_order_share: float = 0.0
    stock_multiplier: int = 1


SCENARIOS = {
    "small": ScenarioConfig(
        name="small",
        bakery_count=1,
        clients_per_bakery=8,
        confectioners_per_bakery=2,
        cashiers_per_bakery=2,
        couriers_per_bakery=2,
        products_per_bakery=6,
        orders_per_bakery=12,
        schedule_days=7,
        max_items_per_order=3,
        delivery_share=0.45,
        commit_every=1,
    ),
    "medium": ScenarioConfig(
        name="medium",
        bakery_count=10,
        clients_per_bakery=20,
        confectioners_per_bakery=2,
        cashiers_per_bakery=2,
        couriers_per_bakery=2,
        products_per_bakery=6,
        orders_per_bakery=30,
        schedule_days=7,
        max_items_per_order=4,
```

```
        delivery_share=0.50,
        commit_every=5,
    ),
    "large": ScenarioConfig(
        name="large",
        bakery_count=1000,
        clients_per_bakery=5,
        confectioners_per_bakery=1,
        cashiers_per_bakery=1,
        couriers_per_bakery=1,
        products_per_bakery=5,
        orders_per_bakery=4,
        schedule_days=5,
        max_items_per_order=3,
        delivery_share=0.40,
        commit_every=50,
    ),
"index_demo": ScenarioConfig(
    name="index_demo",
    bakery_count=100,
    clients_per_bakery=200,
    confectioners_per_bakery=2,
    cashiers_per_bakery=2,
    couriers_per_bakery=2,
    products_per_bakery=7,
    orders_per_bakery=5000,
    schedule_days=7,
    max_items_per_order=5,
    delivery_share=0.15,
    commit_every=5,
    open_order_share=0.98,
    stock_multiplier=300,
```

```
    ),
}

INGREDIENTS = [
    {
        "name": "Мука",
        "description": "Пшеничная мука высшего сорта",
        "cost_per_kg": 70,
        "cost_per_piece": None,
        "calories_per_100g": 364,
        "calories_per_piece": None,
        "shelf_life_days": 180,
        "can_be_weight": True,
        "can_be_piece": False,
    },
    {
        "name": "Сахар",
        "description": "Белый сахар-песок",
        "cost_per_kg": 85,
        "cost_per_piece": None,
        "calories_per_100g": 399,
        "calories_per_piece": None,
        "shelf_life_days": 365,
        "can_be_weight": True,
        "can_be_piece": False,
    },
    {
        "name": "Яйцо",
        "description": "Куриное яйцо категории С1",
        "cost_per_kg": None,
        "cost_per_piece": 12,
        "calories_per_100g": None,
```

```
    "calories_per_piece": 78,
    "shelf_life_days": 25,
    "can_be_weight": False,
    "can_be_piece": True,
},
{
    "name": "Молоко",
    "description": "Молоко 3.2%",
    "cost_per_kg": 95,
    "cost_per_piece": None,
    "calories_per_100g": 60,
    "calories_per_piece": None,
    "shelf_life_days": 7,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Сливки",
    "description": "Сливки 33%",
    "cost_per_kg": 420,
    "cost_per_piece": None,
    "calories_per_100g": 340,
    "calories_per_piece": None,
    "shelf_life_days": 10,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Масло сливочное",
    "description": "Масло 82.5%",
    "cost_per_kg": 750,
    "cost_per_piece": None,
```

```
    "calories_per_100g": 748,
    "calories_per_piece": None,
    "shelf_life_days": 30,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Шоколад темный",
    "description": "Тёмный шоколад 70%",
    "cost_per_kg": 980,
    "cost_per_piece": None,
    "calories_per_100g": 545,
    "calories_per_piece": None,
    "shelf_life_days": 180,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Какао",
    "description": "Порошок какао",
    "cost_per_kg": 650,
    "cost_per_piece": None,
    "calories_per_100g": 228,
    "calories_per_piece": None,
    "shelf_life_days": 365,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Ваниль",
    "description": "Ванильный сахар и экстракт",
    "cost_per_kg": 2500,
```

```
    "cost_per_piece": None,
    "calories_per_100g": 288,
    "calories_per_piece": None,
    "shelf_life_days": 365,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Клубника",
    "description": "Свежая клубника",
    "cost_per_kg": 450,
    "cost_per_piece": None,
    "calories_per_100g": 32,
    "calories_per_piece": None,
    "shelf_life_days": 4,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Малина",
    "description": "Свежая малина",
    "cost_per_kg": 650,
    "cost_per_piece": None,
    "calories_per_100g": 52,
    "calories_per_piece": None,
    "shelf_life_days": 3,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Черника",
    "description": "Свежая черника",
```

```
    "cost_per_kg": 700,
    "cost_per_piece": None,
    "calories_per_100g": 57,
    "calories_per_piece": None,
    "shelf_life_days": 4,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Орех фундук",
    "description": "Очищенный фундук",
    "cost_per_kg": 1100,
    "cost_per_piece": None,
    "calories_per_100g": 628,
    "calories_per_piece": None,
    "shelf_life_days": 180,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Миндаль",
    "description": "Очищенный миндаль",
    "cost_per_kg": 1250,
    "cost_per_piece": None,
    "calories_per_100g": 579,
    "calories_per_piece": None,
    "shelf_life_days": 180,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Творожный сыр",
```

```
    "description": "Сливочный творожный сыр",
    "cost_per_kg": 760,
    "cost_per_piece": None,
    "calories_per_100g": 342,
    "calories_per_piece": None,
    "shelf_life_days": 14,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Маскарпоне",
    "description": "Сыр маскарпоне",
    "cost_per_kg": 1150,
    "cost_per_piece": None,
    "calories_per_100g": 412,
    "calories_per_piece": None,
    "shelf_life_days": 14,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Печенье савоярди",
    "description": "Печенье для тирамису",
    "cost_per_kg": 700,
    "cost_per_piece": 18,
    "calories_per_100g": 380,
    "calories_per_piece": 35,
    "shelf_life_days": 120,
    "can_be_weight": True,
    "can_be_piece": True,
},
{
```

```
    "name": "Кофе",
    "description": "Крепкий кофе для пропитки",
    "cost_per_kg": 1600,
    "cost_per_piece": None,
    "calories_per_100g": 2,
    "calories_per_piece": None,
    "shelf_life_days": 365,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Лимон",
    "description": "Свежий лимон",
    "cost_per_kg": 260,
    "cost_per_piece": 35,
    "calories_per_100g": 29,
    "calories_per_piece": 20,
    "shelf_life_days": 20,
    "can_be_weight": True,
    "can_be_piece": True,
},
{
    "name": "Мёд",
    "description": "Цветочный мед",
    "cost_per_kg": 900,
    "cost_per_piece": None,
    "calories_per_100g": 304,
    "calories_per_piece": None,
    "shelf_life_days": 365,
    "can_be_weight": True,
    "can_be_piece": False,
},
```

```
{
    "name": "Карамель",
    "description": "Солёная карамель",
    "cost_per_kg": 780,
    "cost_per_piece": None,
    "calories_per_100g": 382,
    "calories_per_piece": None,
    "shelf_life_days": 60,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Фисташка",
    "description": "Фисташковая крошка",
    "cost_per_kg": 1500,
    "cost_per_piece": None,
    "calories_per_100g": 560,
    "calories_per_piece": None,
    "shelf_life_days": 180,
    "can_be_weight": True,
    "can_be_piece": False,
},
{
    "name": "Банан",
    "description": "Спелый банан",
    "cost_per_kg": 160,
    "cost_per_piece": 22,
    "calories_per_100g": 89,
    "calories_per_piece": 105,
    "shelf_life_days": 7,
    "can_be_weight": True,
    "can_be_piece": True,
```

```
    },
    {
        "name": "Соль",
        "description": "Пищевая соль",
        "cost_per_kg": 25,
        "cost_per_piece": None,
        "calories_per_100g": 0,
        "calories_per_piece": None,
        "shelf_life_days": 365,
        "can_be_weight": True,
        "can_be_piece": False,
    },
]

UTENSILS = [
    {"name": "Венчик", "description": "Ручной венчик"},
    {"name": "Миксер", "description": "Планетарный миксер"},
    {"name": "Духовка", "description": "Пекарская духовка"},
    {"name": "Кондитерский мешок", "description": "Для крема и начинок"},
    {"name": "Форма для выпечки", "description": "Круглая разъёмная форма"},
    {"name": "Лопатка", "description": "Силиконовая лопатка"},
    {"name": "Весы", "description": "Кухонные электронные весы"},
    {"name": "Миска", "description": "Стальная миска"},
    {"name": "Нож", "description": "Поварской нож"},
    {"name": "Тортовое кольцо", "description": "Кольцо для сборки торта"},
]

PRODUCT_TEMPLATES = [
    {
        "name": "Эклер ванильный",
        "description": "Классический эклер с ванильным кремом",
        "ingredients": [
```

```
            ("Мука", 60, "g", 18),
            ("Яйцо", 2, "pcs", 18),
            ("Молоко", 80, "g", 12),
            ("Сливки", 45, "g", 18),
            ("Сахар", 22, "g", 10),
            ("Масло сливочное", 28, "g", 12),
            ("Ваниль", 3, "g", 12),
        ],
        "toppings": ["Шоколад темный", "Карамель", "Малина"],
        "utensils": ["Венчик", "Миксер", "Духовка", "Кондитерский мешок", "Весы"],
        "replacements": [
            {"group_name": "Замена кремовой основы", "base": "Сливки", "subs": ["Творожный сыр"], "taste_drop_pct": 8},
        ],
    },
    {
        "name": "Наполеон",
        "description": "Слоёный торт с нежным кремом",
        "ingredients": [
            ("Мука", 110, "g", 18),
            ("Масло сливочное", 55, "g", 18),
            ("Молоко", 70, "g", 10),
            ("Сахар", 24, "g", 10),
            ("Яйцо", 1, "pcs", 8),
            ("Ваниль", 2, "g", 8),
            ("Сливки", 35, "g", 14),
            ("Соль", 1, "g", 4),
        ],
        "toppings": ["Карамель", "Клубника"],
        "utensils": ["Венчик", "Духовка", "Форма для выпечки", "Лопатка", "Весы"],
        "replacements": [],
    },
```

```
  {
    "name": "Медовик",
    "description": "Медовые коржи с кремом",
    "ingredients": [
      ("Мука", 90, "g", 20),
      ("Мёд", 30, "g", 20),
      ("Сахар", 20, "g", 10),
      ("Яйцо", 1, "pcs", 10),
      ("Масло сливочное", 24, "g", 10),
      ("Сливки", 40, "g", 20),
      ("Ваниль", 2, "g", 10),
    ],
    "toppings": ["Фисташка", "Карамель"],
    "utensils": ["Венчик", "Духовка", "Лопатка", "Весы", "Миска"],
    "replacements": [
      {"group_name": "Замена кремовой основы", "base": "Сливки", "subs": ["Творожный сыр"], "taste_drop_pct": 10},
    ],
  },
  {
    "name": "Тирамису",
    "description": "Десерт с маскарпоне и кофейной пропиткой",
    "ingredients": [
      ("Маскарпоне", 70, "g", 30),
      ("Печенье савоярди", 4, "pcs", 20),
      ("Кофе", 8, "g", 12),
      ("Сливки", 35, "g", 12),
      ("Сахар", 18, "g", 8),
      ("Какао", 6, "g", 10),
      ("Яйцо", 1, "pcs", 8),
    ],
    "toppings": ["Какао", "Шоколад темный", "Клубника"],
```

```
        "utensils": ["Миксер", "Миска", "Весы", "Лопатка", "Тортовое кольцо"],
        "replacements": [
            {"group_name": "Замена сырной основы", "base": "Маскарпоне", "subs": ["Творожный сыр"], "taste_drop_pct": 12},
        ],
    },
    {
        "name": "Чизкейк",
        "description": "Запечённый чизкейк с ягодным акцентом",
        "ingredients": [
            ("Творожный сыр", 85, "g", 30),
            ("Сливки", 25, "g", 10),
            ("Сахар", 18, "g", 8),
            ("Яйцо", 1, "pcs", 10),
            ("Масло сливочное", 18, "g", 6),
            ("Мука", 24, "g", 8),
            ("Клубника", 25, "g", 18),
            ("Ваниль", 2, "g", 10),
        ],
        "toppings": ["Малина", "Черника", "Карамель"],
        "utensils": ["Миксер", "Духовка", "Форма для выпечки", "Весы", "Лопатка"],
        "replacements": [
            {"group_name": "Замена ягоды", "base": "Клубника", "subs": ["Малина", "Черника"], "taste_drop_pct": 6},
        ],
    },
    {
        "name": "Брауни",
        "description": "Шоколадный брауни с ореховой ноткой",
        "ingredients": [
            ("Шоколад темный", 55, "g", 26),
            ("Масло сливочное", 35, "g", 14),
```

```
            ("Сахар", 26, "g", 10),
            ("Яйцо", 1, "pcs", 10),
            ("Мука", 22, "g", 8),
            ("Какао", 8, "g", 10),
            ("Орех фундук", 16, "g", 12),
            ("Соль", 1, "g", 10),
        ],
        "toppings": ["Карамель", "Фисташка", "Банан"],
        "utensils": ["Миксер", "Духовка", "Форма для выпечки", "Нож", "Весы"],
        "replacements": [
            {"group_name": "Замена ореха", "base": "Орех фундук", "subs": ["Миндаль",
"Фисташка"], "taste_drop_pct": 7},
        ],
    },
    {
        "name": "Тарталетка ягодная",
        "description": "Песочная тарталетка с ягодами и кремом",
        "ingredients": [
            ("Мука", 50, "g", 18),
            ("Масло сливочное", 24, "g", 14),
            ("Сахар", 16, "g", 8),
            ("Яйцо", 1, "pcs", 8),
            ("Сливки", 24, "g", 12),
            ("Клубника", 18, "g", 14),
            ("Малина", 12, "g", 12),
            ("Черника", 10, "g", 8),
            ("Ваниль", 2, "g", 6),
        ],
        "toppings": ["Черника", "Малина", "Карамель"],
        "utensils": ["Духовка", "Форма для выпечки", "Венчик", "Весы", "Лопатка"],
        "replacements": [
```

```
            {"group_name": "Замена ягодной группы", "base": "Клубника", "subs": ["Малина", "Черника"], "taste_drop_pct": 5},
        ],
    },
]

FIRST_NAMES = [
    "Анна", "Мария", "Елена", "Ольга", "Ирина", "Наталья", "Светлана", "Татьяна",
    "Алексей", "Иван", "Дмитрий", "Михаил", "Сергей", "Павел", "Андрей", "Максим",
]

LAST_NAMES = [
    "Иванова", "Петрова", "Сидорова", "Кузнецова", "Соколова", "Смирнова", "Морозова",
    "Фёдорова", "Орлова", "Васильева", "Иванов", "Петров", "Сидоров", "Кузнецов",
    "Соколов", "Смирнов", "Морозов", "Фёдоров", "Орлов", "Васильев",
]

MIDDLE_NAMES = [
    "Александрович", "Сергеевич", "Иванович", "Павлович", "Михайлович", "Дмитриевич",
    "Александровна", "Сергеевна", "Ивановна", "Павловна", "Михайловна", "Дмитриевна",
]

STREET_NAMES = [
    "Арбат", "Тверская", "Сретенка", "Покровка", "Мясницкая", "Большая Никитская",
    "Ленинский проспект", "Профсоюзная", "Кутузовский проспект", "Пречистенка",
    "Маросейка", "Неглинная", "Долгоруковская", "Новослободская", "Садовая-Самотёчная",
]

DISTRICTS = [
    "Центральная", "Сокольники", "Замоскворечье", "Хамовники", "Таганская",
    "Басманная", "Пресненская", "Тверская", "Якиманка", "Даниловская",
```

```
]


STORE_PREFIXES = ["Фермерский рынок", "Гастроном", "Поставщик", "Маркет",
"Лавка"]


ORDER_STATUSES = {
    "pickup": "COMPLETED",
    "cafe": "COMPLETED",
    "delivery_ok": "DELIVERED",
    "delivery_claim": "RETURNED",
}


def money(value):
    """Округление денег до 2 знаков"""
    if not isinstance(value, Decimal):
        value = Decimal(str(value))
    return value.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)


def random_person_name(rng):
    return f"{rng.choice(LAST_NAMES)} {rng.choice(FIRST_NAMES)}
{rng.choice(MIDDLE_NAMES)}"


def phone_by_number(number):
    return f"+7-9{number % 10}{(number // 10) % 10}-{number % 1000:03d}-{(number * 7) %
100:02d}-{(number * 13) % 100:02d}"


def deterministic_password_hash(login):
    return hashlib.sha256(f"demo-password-for-{login}".encode("utf-8")).hexdigest()


def passport_no(index):
    series = 4500 + index // 900000
    number = 100000 + index % 900000
    return f"{series:04d} {number:06d}"
```

```
def bakery_name(index):
    district = DISTRICTS[(index - 1) % len(DISTRICTS)]
    return f"Старый пяточок {district} #{index}"


def bakery_address(index):
    street = STREET_NAMES[(index - 1) % len(STREET_NAMES)]
    house = 1 + (index * 3) % 80
    return f"г. Москва, ул. {street}, д. {house}"


def store_name(index):
    prefix = STORE_PREFIXES[(index - 1) % len(STORE_PREFIXES)]
    district = DISTRICTS[(index - 1) % len(DISTRICTS)]
    return f"{prefix} {district} #{index}"


def pick_work_hours(role_code):
    if role_code == "OWNER":
        return time(10, 0), time(18, 0), time(13, 0), time(14, 0)
    if role_code == "CHEF":
        return time(7, 0), time(15, 0), time(11, 0), time(11, 30)
    if role_code == "CONFECTIONER":
        return time(8, 0), time(16, 0), time(12, 0), time(12, 30)
    if role_code == "CASHIER":
        return time(9, 0), time(18, 0), time(13, 30), time(14, 0)
    return time(10, 0), time(19, 0), time(14, 0), time(14, 30)  # COURIER


def compute_ingredient_cost(ingredient_def, amount_value, amount_unit):
    if amount_unit == "g":
        assert ingredient_def["cost_per_kg"] is not None
        return money(Decimal(str(ingredient_def["cost_per_kg"])) * amount_value /
Decimal("1000"))
    assert ingredient_def["cost_per_piece"] is not None
```

```
    return money(Decimal(str(ingredient_def["cost_per_piece"])) * amount_value)


def compute_ingredient_calories(ingredient_def, amount_value, amount_unit):
    if amount_unit == "g":
        calories = ingredient_def["calories_per_100g"] or 0
        return money(Decimal(str(calories)) * amount_value / Decimal("100"))
    calories = ingredient_def["calories_per_piece"] or 0
    return money(Decimal(str(calories)) * amount_value)


def choose_product_templates(products_per_bakery, bakery_index):
    templates = []
    start = (bakery_index - 1) % len(PRODUCT_TEMPLATES)
    for offset in range(products_per_bakery):
        templates.append(PRODUCT_TEMPLATES[(start + offset) %
len(PRODUCT_TEMPLATES)])
    return templates


TABLES_IN_DELETE_ORDER  = [
    "activity_log",
    "invoice_return_item",
    "order_item_exclusion",
    "order_item_topping",
    "order_item",
    "invoice",
    "customer_order",
    "replacement_group_item",
    "replacement_group",
    "allowed_topping",
    "price_list_item_ingredient",
    "price_list_item",
    "price_list",
    "product_utensil",
```

```
    "store_utensil_assortment",
    "store_ingredient_assortment",
    "bakery_utensil_inventory",
    "bakery_ingredient_stock",
    "product",
    "work_schedule",
    "employee",
    "client",
    "store",
    "utensil",
    "ingredient",
    "bakery",
]

def reset_database(cur):
    for table_name in TABLES_IN_DELETE_ORDER:
        cur.execute(f"DELETE FROM {table_name}")

def insert_many_returning(cur, sql, rows):
    if not rows:
        return []
    execute_values(cur, sql, rows, page_size=1000)
    return cur.fetchall()

def insert_reference_data(cur):
    ingredient_rows = [
        (
            item["name"],
            item["description"],
            item["cost_per_kg"],
            item["cost_per_piece"],
            item["calories_per_100g"],
```

```
        item["calories_per_piece"],
        item["shelf_life_days"],
        item["can_be_weight"],
        item["can_be_piece"],
    )
    for item in INGREDIENTS
]
ingredient_res = insert_many_returning(
    cur,
    """
    INSERT INTO ingredient (
        name, description, cost_per_kg, cost_per_piece,
        calories_per_100g, calories_per_piece, shelf_life_days,
        can_be_weight, can_be_piece
    ) VALUES %s
    RETURNING ingredient_id, name
    """,
    ingredient_rows,
)
ingredient_ids = {name: ingredient_id for ingredient_id, name in ingredient_res}
ingredient_defs = {item["name"]: item for item in INGREDIENTS}

utensil_rows = [(item["name"], item["description"]) for item in UTENSILS]
utensil_res = insert_many_returning(
    cur,
    """
    INSERT INTO utensil (name, description)
    VALUES %s
    RETURNING utensil_id, name
    """,
    utensil_rows,
)
```

```
    utensil_ids = {name: utensil_id for utensil_id, name in utensil_res}


    store_rows = []
    for i in range(1, 6):
        store_rows.append((store_name(i), bakery_address(1000 + i), phone_by_number(7000 +
i)))
    store_res = insert_many_returning(
        cur,
        """
        INSERT INTO store (name, address, phone)
        VALUES %s
        RETURNING store_id, name
        """,
        store_rows,
    )
    store_ids = [store_id for store_id, _ in store_res]


    store_ingredient_rows = []
    for store_idx, store_id in enumerate(store_ids, start=1):
        for ingredient in INGREDIENTS:
            price_kg = None
            if ingredient["cost_per_kg"] is not None:
                coef = Decimal("1.15") + Decimal(str((store_idx - 1) * 0.03))
                price_kg = money(Decimal(str(ingredient["cost_per_kg"])) * coef)
            price_piece = None
            if ingredient["cost_per_piece"] is not None:
                coef = Decimal("1.12") + Decimal(str((store_idx - 1) * 0.02))
                price_piece = money(Decimal(str(ingredient["cost_per_piece"])) * coef)
            stock_g = Decimal("50000") if ingredient["can_be_weight"] else Decimal("0")
            stock_pcs = 400 if ingredient["can_be_piece"] else 0
            store_ingredient_rows.append((
                store_id,
```

```
            ingredient_ids[ingredient["name"]],
            price_kg,
            price_piece,
            stock_g,
            stock_pcs,
        ))
execute_values(
    cur,
    """
    INSERT INTO store_ingredient_assortment (
        store_id, ingredient_id, price_per_kg, price_per_piece, stock_qty_g, stock_qty_pcs
    ) VALUES %s
    """,
    store_ingredient_rows,
    page_size=1000,
)

store_utensil_rows = []
for store_idx, store_id in enumerate(store_ids, start=1):
    for utensil_id, utensil_name in utensil_res:
        base_price = Decimal("350") + Decimal(str((utensil_id % 7) * 90))
        store_utensil_rows.append((
            store_id,
            utensil_id,
            money(base_price * (Decimal("1.10") + Decimal(str(store_idx * 0.01)))),
            20 + store_idx * 2,
        ))
execute_values(
    cur,
    """
    INSERT INTO store_utensil_assortment (store_id, utensil_id, price_amount, stock_qty)
    VALUES %s
```

```
        """,
        store_utensil_rows,
        page_size=1000,
    )

    return {
        "ingredient_ids": ingredient_ids,
        "ingredient_defs": ingredient_defs,
        "utensil_ids": utensil_ids,
        "store_ids": store_ids,
    }

def create_bakeries(cur, config):
    rows = []
    for i in range(1, config.bakery_count + 1):
        rows.append((
            bakery_name(i),
            bakery_address(i),
            time(8, 0),
            time(21, 0),
            money(120000 + i * 1500),
            phone_by_number(1000 + i),
        ))
    result = insert_many_returning(
        cur,
        """
        INSERT INTO bakery (name, address, open_time, close_time, rent_cost, phone)
        VALUES %s
        RETURNING bakery_id
        """,
        rows,
    )
```

```
    return [item[0] for item in result]


def create_clients_for_bakery(cur, bakery_index, config, rng):
    rows = []
    for local_idx in range(1, config.clients_per_bakery + 1):
        serial = bakery_index * 1000 + local_idx
        full_name = random_person_name(rng)
        email = f"client_{bakery_index}_{local_idx}@example.test"
        rows.append((
            full_name,
            phone_by_number(200000 + serial),
            email,
            f"г. Москва, ул. {STREET_NAMES[serial % len(STREET_NAMES)]}, д. {10 + serial
% 90}",
        ))
    result = insert_many_returning(
        cur,
        """
        INSERT INTO client (full_name, phone, email, default_delivery_address)
        VALUES %s
        RETURNING client_id
        """,
        rows,
    )
    return [item[0] for item in result]


def create_employees_for_bakery(cur, bakery_id, bakery_index, config, rng):
    rows = []
    role_plan = [
        ("OWNER", 1),
        ("CHEF", 1),
        ("CONFECTIONER", config.confectioners_per_bakery),
```

```
    ("CASHIER", config.cashiers_per_bakery),
    ("COURIER", config.couriers_per_bakery),
]

role_salary = {
    "OWNER": (0, 180000, 0, 0),
    "CHEF": (900, 30000, 0, 150),
    "CONFECTIONER": (550, 12000, 0, 160),
    "CASHIER": (380, 18000, 0, 150),
    "COURIER": (250, 35000, 4.5, 140),
}

employee_counter = 0
for role_code, amount in role_plan:
    for role_idx in range(1, amount + 1):
        employee_counter += 1
        serial = bakery_index * 1000 + employee_counter
        login = f"{role_code.lower()}_{bakery_index}_{role_idx}"
        hourly_rate, fixed_salary, bonus_pct, min_hours = role_salary[role_code]
        rows.append((
            bakery_id,
            random_person_name(rng),
            passport_no(serial),
            role_code,
            money(hourly_rate),
            money(fixed_salary),
            money(bonus_pct),
            Decimal(str(min_hours)),
            login,
            deterministic_password_hash(login),
            True,
            date(2023, 1, 1) + timedelta(days=(serial % 500)),
```

```
            ))

    result = insert_many_returning(
        cur,
        """
        INSERT INTO employee (
            bakery_id, full_name, passport_no, role_code,
            hourly_rate, fixed_monthly_salary, courier_bonus_pct,
            min_hours_per_month, login, password_hash, is_active, hire_date
        ) VALUES %s
        RETURNING employee_id, role_code, login
        """,
        rows,
    )

    employees_by_role = {}
    for employee_id, role_code, _login in result:
        employees_by_role.setdefault(role_code, []).append(employee_id)
    return employees_by_role


def create_schedule_for_bakery(cur, bakery_id, employees_by_role, config):
    rows = []
    start_date = date.today() - timedelta(days=3)
    for role_code, employee_ids in employees_by_role.items():
        for employee_id in employee_ids:
            for day_offset in range(config.schedule_days):
                work_date = start_date + timedelta(days=day_offset)
                if work_date.weekday() == 6 and role_code in {"OWNER", "CASHIER"}:
                    continue
                work_start, work_end, break_start, break_end = pick_work_hours(role_code)
                rows.append((
                    employee_id,
```

```
                bakery_id,
                work_date,
                work_date.isoweekday(),
                work_start,
                work_end,
                break_start,
                break_end,
            ))
    execute_values(
        cur,
        """
        INSERT INTO work_schedule (
            employee_id, bakery_id, work_date, day_of_week,
            work_start_time, work_end_time,
            break_start_time, break_end_time
        ) VALUES %s
        """,
        rows,
        page_size=1000,
    )

def create_stocks_for_bakery(cur, bakery_id, bakery_index, ref, config, rng):
    ingredient_rows = []
    stock_multiplier = Decimal(str(config.stock_multiplier))
    for ingredient in INGREDIENTS:
        factor = Decimal(str(1 + (bakery_index % 5) * 0.15))
        stock_g = Decimal("0")
        if ingredient["can_be_weight"]:
            stock_g = (
                Decimal("5000") + Decimal(str(rng.randint(0, 7000)))
            ) * factor * stock_multiplier
        stock_pcs = 0
```

```
            if ingredient["can_be_piece"]:
                stock_pcs = int(
                    (40 + rng.randint(0, 80))
                    * float(factor)
                    * config.stock_multiplier
                )
            daily_g = Decimal("0")
            if ingredient["can_be_weight"]:
                daily_g = money(Decimal("150") + Decimal(str(rng.randint(0, 120))))
            daily_pcs = 0
            if ingredient["can_be_piece"]:
                daily_pcs = 5 + rng.randint(0, 8)
            ingredient_rows.append((
                bakery_id,
                ref["ingredient_ids"][ingredient["name"]],
                stock_g,
                stock_pcs,
                daily_g,
                daily_pcs,
            ))
        execute_values(
            cur,
            """
            INSERT INTO bakery_ingredient_stock (
                bakery_id, ingredient_id, stock_qty_g, stock_qty_pcs,
                avg_daily_consumption_g, avg_daily_consumption_pcs
            ) VALUES %s
            """,
            ingredient_rows,
            page_size=1000,
        )
```

```
    utensil_rows = []
    for utensil_name, utensil_id in ref["utensil_ids"].items():
        qty = 1 + (bakery_index + utensil_id) % 6
        condition = "good" if qty >= 2 else "reserve"
        utensil_rows.append((bakery_id, utensil_id, qty, condition))
    execute_values(
        cur,
        """
        INSERT INTO bakery_utensil_inventory (bakery_id, utensil_id, qty_available,
condition_status)
        VALUES %s
        """,
        utensil_rows,
        page_size=1000,
    )

def create_products_and_pricelist(cur, bakery_id, bakery_index, chef_id, config, ref):
    templates = choose_product_templates(config.products_per_bakery, bakery_index)

    product_rows = []
    for template in templates:
        product_rows.append((chef_id, template["name"], template["description"], True))

    product_result = insert_many_returning(
        cur,
        """
        INSERT INTO product (chef_id, name, recipe_description, is_active)
        VALUES %s
        RETURNING product_id, name
        """,
        product_rows,
    )
```

```
    product_id_sequence = [row[0] for row in product_result]


    product_utensil_rows = []
    for product_id, template in zip(product_id_sequence, templates):
        for utensil_name in template["utensils"]:
            product_utensil_rows.append((product_id, ref["utensil_ids"][utensil_name], f"Основная утварь для {template['name']}"))
    execute_values(
        cur,
        """
        INSERT INTO product_utensil (product_id, utensil_id, usage_note)
        VALUES %s
        """,
        product_utensil_rows,
        page_size=1000,
    )


    price_list_name = f"Основной прайс {bakery_index}"
    cur.execute(
        """
        INSERT INTO price_list (bakery_id, name, valid_from, valid_to, is_active)
        VALUES (%s, %s, %s, %s, %s)
        RETURNING price_list_id
        """,
        (bakery_id, price_list_name, date.today() - timedelta(days=30), None, True),
    )
    price_list_id = cur.fetchone()[0]


    catalog = []
    for template, product_id in zip(templates, product_id_sequence):
        recipe_cost = Decimal("0")
```

```
        recipe_calories = Decimal("0")
        for ingredient_name, amount_value, amount_unit, _taste in template["ingredients"]:
            ingredient_def = ref["ingredient_defs"][ingredient_name]
            recipe_cost += compute_ingredient_cost(ingredient_def, Decimal(str(amount_value)),
amount_unit)
            recipe_calories += compute_ingredient_calories(ingredient_def,
Decimal(str(amount_value)), amount_unit)

        margin = Decimal("2.70") + Decimal(str((bakery_index % 4) * 0.15))
        sale_price = money(recipe_cost * margin + Decimal("30"))
        prep_time = 15 + len(template["ingredients"]) * 3
        calories = money(recipe_calories)
        item_code = f"B{bakery_index:04d}-P{product_id:05d}"

        cur.execute(
            """
            INSERT INTO price_list_item (
                price_list_id, product_id, item_code, sale_price, prep_time_minutes, calories_kcal
            ) VALUES (%s, %s, %s, %s, %s, %s)
            RETURNING price_list_item_id
            """,
            (price_list_id, product_id, item_code, sale_price, prep_time, calories),
        )
        price_list_item_id = cur.fetchone()[0]

        ingredient_rows = []
        ingredient_names_for_item = []
        for ingredient_name, amount_value, amount_unit, taste_pct in template["ingredients"]:
            ingredient_id = ref["ingredient_ids"][ingredient_name]
            ingredient_names_for_item.append(ingredient_name)
            ingredient_rows.append((
                price_list_item_id,
```

```
                ingredient_id,
                Decimal(str(amount_value)),
                amount_unit,
                Decimal(str(taste_pct)),
            ))
        execute_values(
            cur,
            """
            INSERT INTO price_list_item_ingredient (
                price_list_item_id, ingredient_id, amount_value, amount_unit, taste_contribution_pct
            ) VALUES %s
            """,
            ingredient_rows,
            page_size=1000,
        )

        topping_options = []
        topping_rows = []
        for topping_name in template["toppings"]:
            ingredient_def = ref["ingredient_defs"][topping_name]
            ingredient_id = ref["ingredient_ids"][topping_name]
            if ingredient_def["can_be_weight"]:
                default_amount = Decimal("12")
                amount_unit = "g"
                extra_price = compute_ingredient_cost(ingredient_def, default_amount, amount_unit)
* Decimal("2.3")
            else:
                default_amount = Decimal("1")
                amount_unit = "pcs"
                extra_price = compute_ingredient_cost(ingredient_def, default_amount, amount_unit)
* Decimal("2.1")
            taste_drop = Decimal("4") + Decimal(str((ingredient_id + bakery_index) % 7))
```

```
            extra_price = money(extra_price + Decimal("10"))
            topping_rows.append((
                price_list_item_id,
                ingredient_id,
                default_amount,
                amount_unit,
                extra_price,
                taste_drop,
            ))
            topping_options.append({
                "ingredient_name": topping_name,
                "ingredient_id": ingredient_id,
                "default_amount": default_amount,
                "amount_unit": amount_unit,
                "extra_price": extra_price,
                "taste_drop_pct": taste_drop,
            })
        execute_values(
            cur,
            """
            INSERT INTO allowed_topping (
                price_list_item_id, ingredient_id, default_amount, amount_unit, extra_price,
taste_drop_pct
            ) VALUES %s
            """,
            topping_rows,
            page_size=1000,
        )

        replacement_rows = []
        for replacement_group in template["replacements"]:
            cur.execute(
```

```
            """
            INSERT INTO replacement_group (price_list_item_id, group_name, comment)
            VALUES (%s, %s, %s)
            RETURNING replacement_group_id
            """,
            (
                price_list_item_id,
                replacement_group["group_name"],
                f"Группа замен для {template['name']}",
            ),
        )
        replacement_group_id = cur.fetchone()[0]
        base_ingredient_id = ref["ingredient_ids"][replacement_group["base"]]
        for sub_name in replacement_group["subs"]:
            replacement_rows.append((
                replacement_group_id,
                base_ingredient_id,
                ref["ingredient_ids"][sub_name],
                Decimal(str(replacement_group["taste_drop_pct"])),
            ))
    if replacement_rows:
        execute_values(
            cur,
            """
            INSERT INTO replacement_group_item (
                replacement_group_id, base_ingredient_id, substitute_ingredient_id, taste_drop_pct
            ) VALUES %s
            """,
            replacement_rows,
            page_size=1000,
        )
```

```
        catalog.append({
            "price_list_item_id": price_list_item_id,
            "product_id": product_id,
            "product_name": template["name"],
            "item_code": item_code,
            "base_price": sale_price,
            "calories": calories,
            "ingredients": ingredient_names_for_item,
            "ingredient_ids": [ref["ingredient_ids"][name] for name in ingredient_names_for_item],
            "toppings": topping_options,
        })

    return catalog


def build_order_model(rng, catalog, max_items_per_order, delivery_share):
    order_type = "DELIVERY" if rng.random() < delivery_share else rng.choice(["PICKUP",
"CAFE"])
    items = []
    total_amount = Decimal("0")

    item_count = rng.randint(1, max_items_per_order)
    for _ in range(item_count):
        product = rng.choice(catalog)
        quantity = rng.randint(1, 3)
        toppings = []
        exclusions = []
        taste_left = Decimal("100")
        toppings_price = Decimal("0")

        if product["toppings"] and rng.random() < 0.65:
            topping_count = 1 if rng.random() < 0.75 else min(2, len(product["toppings"]))
            for topping in rng.sample(product["toppings"], topping_count):
```

```
            if taste_left - topping["taste_drop_pct"] < Decimal("55"):
                continue
            toppings.append(topping)
            taste_left -= topping["taste_drop_pct"]
            toppings_price += Decimal(str(topping["extra_price"]))

    if product["ingredient_ids"] and rng.random() < 0.15:
        candidate_exclusion_id = rng.choice(product["ingredient_ids"])
        # Исключение одного ингредиента считаем более болезненным для вкуса,
        # поэтому применяем только если не выходим ниже 50%.
        exclusion_penalty = Decimal("18")
        if taste_left - exclusion_penalty >= Decimal("50"):
            taste_left -= exclusion_penalty
            exclusions.append(candidate_exclusion_id)

    line_total = money((Decimal(str(product["base_price"])) + toppings_price) * quantity)
    total_amount += line_total
    items.append({
        "price_list_item_id": product["price_list_item_id"],
        "quantity": quantity,
        "base_price": Decimal(str(product["base_price"])),
        "toppings_price": money(toppings_price),
        "line_total": line_total,
        "final_taste_pct": taste_left,
        "toppings": toppings,
        "exclusions": exclusions,
    })

return {
    "order_type": order_type,
    "total_amount": money(total_amount),
    "items": items,
```

```
    }


def create_orders_for_bakery(cur, bakery_id, bakery_index, client_ids, employees_by_role,
catalog, config, rng):
    cashiers = employees_by_role["CASHIER"]
    couriers = employees_by_role["COURIER"]


    for order_no in range(1, config.orders_per_bakery + 1):
        model = build_order_model(rng, catalog, config.max_items_per_order,
config.delivery_share)
        order_dt = datetime.now() - timedelta(days=rng.randint(0, 12), hours=rng.randint(0, 10))
        client_id = rng.choice(client_ids)
        cashier_id = rng.choice(cashiers)


        initial_status = ORDER_STATUSES["pickup"] if model["order_type"] != "DELIVERY"
else ORDER_STATUSES["delivery_ok"]
        order_status = initial_status
        if rng.random() < config.open_order_share:
            order_status = rng.choice([
                "CREATED",
                "PAID",
                "IN_PROGRESS",
                "CANCELLED",
            ])
        cur.execute(
            """
            INSERT INTO customer_order (
                client_id, bakery_id, cashier_id, order_datetime,
                order_type, status, order_total_amount
            ) VALUES (%s, %s, %s, %s, %s, %s, %s)
            RETURNING order_id
            """,
            (
```

```
        client_id,
        bakery_id,
        cashier_id,
        order_dt,
        model["order_type"],
        order_status,
        model["total_amount"],
    ),
)
order_id = cur.fetchone()[0]

order_item_ids = []
for item in model["items"]:
    cur.execute(
        """
        INSERT INTO order_item (
            order_id, price_list_item_id, quantity,
            base_price, toppings_price, line_total_amount, final_taste_pct
        ) VALUES (%s, %s, %s, %s, %s, %s, %s)
        RETURNING order_item_id
        """,
        (
            order_id,
            item["price_list_item_id"],
            item["quantity"],
            item["base_price"],
            item["toppings_price"],
            item["line_total"],
            item["final_taste_pct"],
        ),
    )
    order_item_id = cur.fetchone()[0]
```

```
        order_item_ids.append(order_item_id)

        topping_rows = []
        for topping in item["toppings"]:
            topping_rows.append((
                order_item_id,
                topping["ingredient_id"],
                topping["default_amount"],
                topping["amount_unit"],
                topping["extra_price"],
                topping["taste_drop_pct"],
            ))
        if topping_rows:
            execute_values(
                cur,
                """
                INSERT INTO order_item_topping (
                    order_item_id, ingredient_id, amount_value, amount_unit,
                    extra_price, taste_drop_pct_applied
                ) VALUES %s
                """,
                topping_rows,
                page_size=1000,
            )

        exclusion_rows = [(order_item_id, ingredient_id, order_dt) for ingredient_id in
item["exclusions"]]
        if exclusion_rows:
            execute_values(
                cur,
                """
                INSERT INTO order_item_exclusion (order_item_id, ingredient_id, created_at)
```

```
            VALUES %s
            """,
            exclusion_rows,
            page_size=1000,
        )

if model["order_type"] == "DELIVERY":
    courier_id = rng.choice(couriers)
    has_claim = rng.random() < 0.07
    recipient_signed = not has_claim
    delivery_due_at = order_dt + timedelta(hours=2 + rng.randint(0, 3))
    delivered_at = delivery_due_at + timedelta(minutes=rng.randint(-25, 40))
    claim_text = None
    if has_claim:
        claim_text = rng.choice([
            "Повреждена упаковка десерта",
            "Крем сместился при доставке",
            "Изделие не соответствует ожиданиям по оформлению",
        ])

    cur.execute(
        """
        INSERT INTO invoice (
            order_id, courier_employee_id, recipient_fio,
            delivery_address, delivery_due_at, delivered_at,
            recipient_signed, claim_text, has_claim
        ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        RETURNING invoice_id
        """,
        (
            order_id,
            courier_id,
```

```
                random_person_name(rng),
                f"г. Москва, ул. {STREET_NAMES[(bakery_index + order_no) %
len(STREET_NAMES)]}, д. {20 + order_no}",
                delivery_due_at,
                delivered_at,
                recipient_signed,
                claim_text,
                has_claim,
            ),
        )
        invoice_id = cur.fetchone()[0]

        if has_claim:
            cur.execute(
                "UPDATE customer_order SET status = %s WHERE order_id = %s",
                (ORDER_STATUSES["delivery_claim"], order_id),
            )
            returned_item_count = max(1, len(order_item_ids) // 2)
            returned_rows = []
            for order_item_id in order_item_ids[:returned_item_count]:
                returned_rows.append((invoice_id, order_item_id, True, "Возврат по претензии
клиента"))
            execute_values(
                cur,
                """
                INSERT INTO invoice_return_item (invoice_id, order_item_id, is_returned,
return_reason)
                VALUES %s
                """,
                returned_rows,
                page_size=1000,
            )
```

```
def create_logs_for_bakery(cur, bakery_id, bakery_index, employees_by_role, orders_created):
    owner_id = employees_by_role["OWNER"][0]
    chef_id = employees_by_role["CHEF"][0]
    cashier_id = employees_by_role["CASHIER"][0]
    courier_id = employees_by_role["COURIER"][0]


    log_rows = [
        (owner_id, "CREATE_PRICE_LIST", datetime.now(), f"Создан основной прайс для bakery_id={bakery_id}"),
        (chef_id, "CREATE_RECIPE_SET", datetime.now(), f"Заведены базовые рецептуры для кондитерской #{bakery_index}"),
        (cashier_id, "CREATE_ORDERS", datetime.now(), f"Создано заказов: {orders_created} в bakery_id={bakery_id}"),
        (courier_id, "PROCESS_DELIVERIES", datetime.now(), f"Обработаны доставки по bakery_id={bakery_id}"),
        (owner_id, "UPDATE_STOCKS", datetime.now(), f"Заполнены стартовые остатки и учётная утварь по bakery_id={bakery_id}"),
    ]
    execute_values(
        cur,
        """
        INSERT INTO activity_log (employee_id, action_type, action_at, info)
        VALUES %s
        """,
        log_rows,
        page_size=1000,
    )


def populate_database(conn, config, seed, reset):
    rng = random.Random(seed)
    with conn.cursor() as cur:
        if reset:
            print("[1/4] Очистка таблиц")
```

```
    reset_database(cur)
    conn.commit()

print("[2/4] Заполнение общих справочников")
ref = insert_reference_data(cur)
bakery_ids = create_bakeries(cur, config)
conn.commit()

print(f"[3/4] Генерация данных по сценарию '{config.name}'")
for idx, bakery_id in enumerate(bakery_ids, start=1):
    client_ids = create_clients_for_bakery(cur, idx, config, rng)
    employees_by_role = create_employees_for_bakery(cur, bakery_id, idx, config, rng)
    create_schedule_for_bakery(cur, bakery_id, employees_by_role, config)
    create_stocks_for_bakery(cur, bakery_id, idx, ref, config, rng)
    catalog = create_products_and_pricelist(
        cur,
        bakery_id,
        idx,
        employees_by_role["CHEF"][0],
        config,
        ref,
    )
    create_orders_for_bakery(
        cur,
        bakery_id,
        idx,
        client_ids,
        employees_by_role,
        catalog,
        config,
        rng,
    )
```

```
            create_logs_for_bakery(cur, bakery_id, idx, employees_by_role,
config.orders_per_bakery)


            if idx % config.commit_every == 0:
                conn.commit()
                print(f"готово кондитерских: {idx}/{config.bakery_count}")


        conn.commit()
        print("[4/4] Готово")


def parse_args():
    parser = argparse.ArgumentParser(description="Заполнить БД кондитерской тестовыми
данными")
    parser.add_argument(
        "--scenario",
        choices=sorted(SCENARIOS.keys()),
        default="small",
        help=f"Сценарий заполнения: {', '.join(sorted(SCENARIOS.keys()))}",
    )
    parser.add_argument(
        "--dsn",
        default=None,
        help="Строка подключения psycopg2. Если не указана, используются PG*
переменные окружения.",
    )
    parser.add_argument(
        "--seed",
        type=int,
        default=42,
        help="Начальное значение генератора случайных чисел. Нужен для
воспроизводимости.",
    )
    parser.add_argument(
```

```
        "--reset",
        action="store_true",
        help="Перед заполнением очистить таблицы через DELETE.",
    )
    return parser.parse_args()


def main():
    args = parse_args()
    config = SCENARIOS[args.scenario]


    conn = psycopg2.connect(args.dsn) if args.dsn else psycopg2.connect()
    try:
        conn.autocommit = False
        populate_database(conn, config, args.seed, args.reset)
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()


if __name__ == "__main__":
    main()
```

Инструкция по запускку:

Запуск программы:

python main.py --scenario small --reset --dsn "host=localhost port=5432 dbname=confectionery_db user=postgres password=postgres"

--scenario – вариант заполнения базы, опции: small, medium, large, index_demo
--reset – очистка базы перед заполнением
--dsn – данные для подключения к базе